

Greedy Yaklaşım $\rightarrow$ Sırt Çantası $\leftarrow$

Dinamik Programlama $\rightarrow$ Bottom Up çalışır

$\rightarrow$ En iyi çözümü garanti eder

$\rightarrow$ Yaklaşım çözüm yeterli ise greedy $\rightarrow$ görülebilir

$\rightarrow$ Greedy, kesirli sırt çantası problemi için kullanılabilir.

$S \rightarrow$ element kümesi

$w_i \rightarrow$ elementin ağırlığı

$i \rightarrow$ temsil edilen element

$x_i \rightarrow$ elementin bir kuantı

$b_i \rightarrow$ elementin değeri

$W \rightarrow$ çanta kapasitesi

$\left(\frac{x_i}{w_i}\right) \cdot b_i \rightarrow$ çantaya koyduğumuz elementin değeri

$\rightarrow \frac{b_i}{w_i} = v_i$ değerlerine göre elementler büyüden küçüğe

sıralanır. En büyüden başlayarak çanta dolana kadar, çantaya konulur.

* S öğesi yığın (heap) tabanlı PQ kullanılır.

$\hookrightarrow$ Her seçim $O(\log n)$

Bu problem toplam olarak $O(n \log n)$ zaman maliyetindedir.

* Brute force da koreed maliyetliydi? $\xrightarrow{\text{maksimum}} \xrightarrow{\text{Daha az da olabilir.}}$

▽ Büyükten küçüğe sıralanacak birkaç bir maliyet. (Sort Descending)

○ Heap Sort, Selection Sort gibi algoritmalar ile sıralama yapılır.

~~○~~ $W = 100$ $S \rightarrow 50, 50, 40, 30, 20$
 $V \rightarrow 80, 80, 70, 30, 10$

1. Yöğünle her bir $\rightarrow \frac{80}{50} = 1.6, \frac{80}{50} = 1.6, \frac{70}{40} = 1.75, \frac{30}{30} = 1, \frac{10}{20} = 0.5$

2. Sıralama $\rightarrow 40, 50, 50, 30, 20$

3. Çarpma sonucu $\rightarrow \sqrt{\quad} \sqrt{\quad} \frac{1}{5}, \dots$

$70 + 80 + 80 \frac{1}{5} = 150 + 16 = 166 \rightarrow$ en fazla
 net değer?

~~○~~ $I = I_1, I_2, I_3, I_4, I_5$ $W = 60$

$W = (5, 10, 20, 30, 40)$

$V = (30, 20, 100, 90, 160)$

1 $\rightarrow$ Yöğünle $= 6, 2, 5, 3, 4 \rightarrow \frac{V}{W}$

2 $\rightarrow$ Sıralama $5, 20, 40, 30, 10 \rightarrow$ Azaltık

3 $\rightarrow$ Çarpma sonucu $\sqrt{\quad} \sqrt{\quad} \frac{35}{100},$
 $\downarrow \quad \downarrow$
 $30 \quad 100 \quad 160 \cdot \frac{7}{8} \rightarrow 130 + 140 = 270 \rightarrow$ en fazla net değer?

~~Soru~~

$I = A \quad B \quad C$

$W = 2 \quad 2 \quad 3$

$V = 100 \quad 10 \quad 120$

1 $\rightarrow$ Yöğünle

$A \rightarrow 50$

$B \rightarrow 5$

$C \rightarrow 40$

2 $\rightarrow$ Sıralama

$A < B$

3 $\rightarrow$ Çarpma

$\frac{2}{\sqrt{\quad}} \frac{3}{\sqrt{\quad}} \frac{120}{2} \rightarrow 100 + 120 + \frac{10}{2} = \frac{225}{7}$

Maliyet

- 1-) $b_i = \frac{v_i}{w_i} \rightarrow$ yoğunluk hesabı $O(n)$
- 2-) Sıralama $O(n \log n)$ (HeapSort) ile
- 3-) Seçim ?

Brute Force = n nesne için $O(2^n)$ tespit

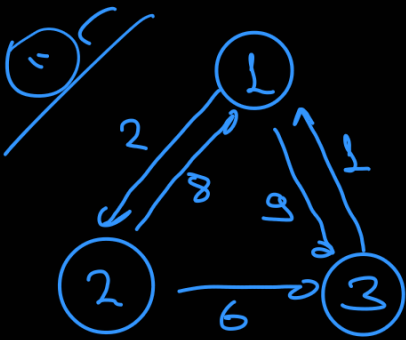
ve n kere de en büyük elemanı için zaman,

$O(n \cdot 2^n)$ zaman maliyeti olur.

Floyd Warshall

→ Yönlü graflarda düğümler arası en kısa mesafeyi tespit etmek için kullanılır.

Brute Force = Düğümler arası yolların hepsinin ayrı ayrı deneyerek en kısa olanı bulması gerekiyor. $O(2^n)$ 'li bir mükemmel maliyete sahiptir.



→ Verilen grafin tablosu yapılır.

$$D_0 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & - & 0 \end{bmatrix}$$

→ Birinci değeri, eğer önce 1'e sonra başka bir değeri değilse 1'e yazılır.

$$D_1 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

→ Aynı şekilde 2. düğüme ve sonra 3. düğüme göre yapılır.

$$D_2 = \begin{bmatrix} 0 & 2 & 8 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

$$D_3 = \begin{bmatrix} 0 & 2 & 8 \\ 7 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

→ Tüm düğümler için işlem yapıldıca nihai sonucu ulaşılmış olur.

Maliyet = Her düğüm için,
1 düğüm için her satır
1 satır için her sütun

Zaman Maliyeti $O(n^3)$

teknik $n^3 \rightarrow$ Değişken maliyet

Bellek Maliyeti = $O(n^2)$ (Bir matris)
teknik n^2

Polinom Hesabı

Binary Exponentiation

→ Bir transfer and conquer yöntemidir bu.

a^n hesaplamak için a^n 'i dönüştürüyoruz.

n → binary olarak ifade ediyoruz.

Left to Right

a → sabit sayı

$b(n)$ → binary gösterim dizisi

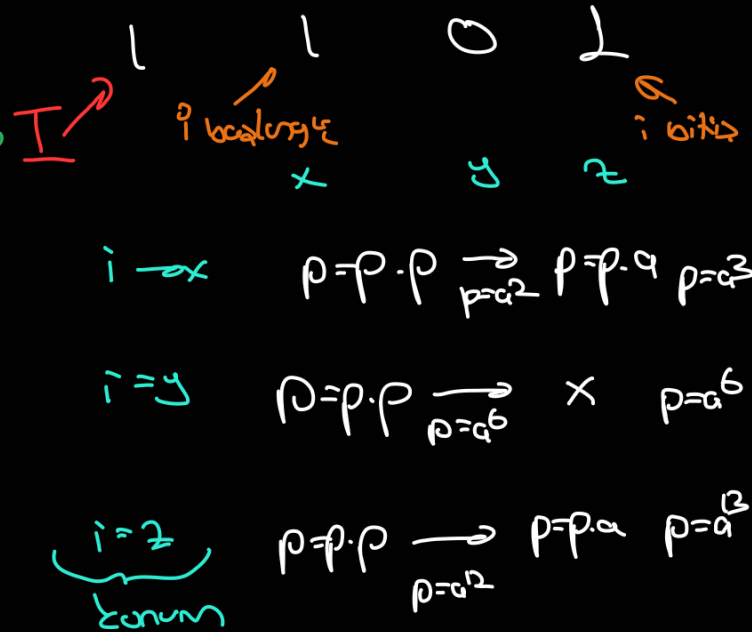
$p = a$

For $i = I-1$ 'den 0 'a

$p = p \times p$

if $b_i = 1$

$p = p \cdot a$



Maliyet = $|I| = \log_2 n$

For döngüsü $(\log_2 n) - 1$ defa çalışır.

$(\log_2 n)$ defa karesi alma

$(\log_2 n) - 1$ defa $p = p \cdot p$ işlemleri

$(\log_2 n) - 1$ max defa $p = p \cdot a$ işlemleri



Hesap Maliyet = $(\log_2 n) - 1$ ile $2 \cdot (\log_2 n) - 1$ arasında

Right to Left

$a \rightarrow$ sayı $b[n] \rightarrow$ binary bit dizisi; " n " ile n

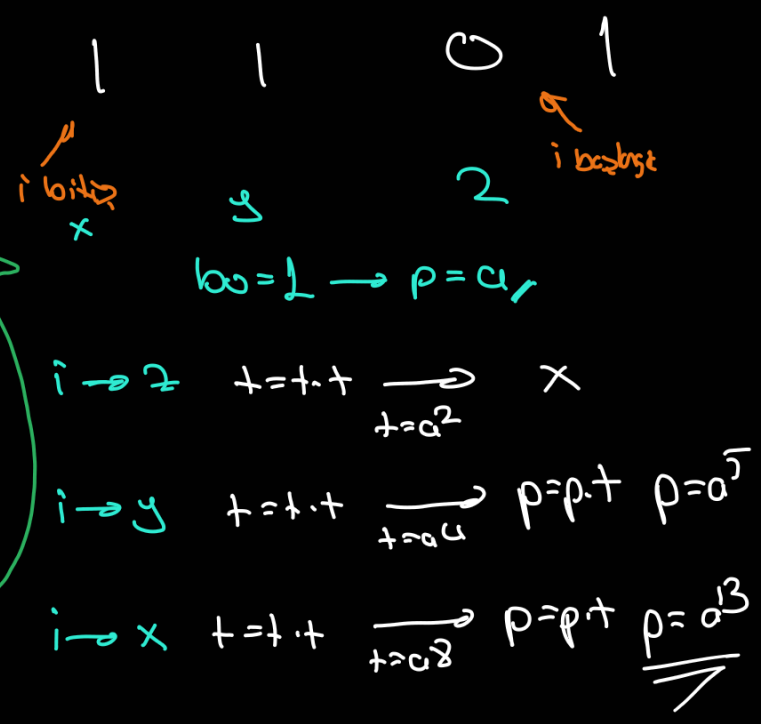
$t = a$

if ($b_0 = 1$) ise $p = a$ } Soğuk
else ise $p = 1$ } en kötü
} basılır

for $i = 1$ 'den I 'ya kadar

$t = t \cdot t$

if ($b_i = 1$) ise
 $p = p \cdot t$



Maliyet $|I| = \log_2 n$

for döngüsü $\log_2 n - 1$ defa çalışır.

$(\log_2 n)$ defa kontrolünce $i = 1$ 'den I 'ya

$(\log_2 n) - 1$ defa $t = t \cdot t$ yapılır

$(\log_2 n) - 1$ max defa $p = p \cdot t$ yapılır

★ Hesap Maliyeti = $(\log_2 n) - 1$ ile $2 \cdot (\log_2 n) - 1$ arasında
en iyi durum en kötü durum

B 2-3 Ağacı

- maksimum 2 düğüm ve 3 çocuk olabilir.
- 3 çocuk var ise bir düğümde 2 eleman olabilir.
- yükseklik dengelidir.

Maliyet $\log_2(n+1)-1 \rightarrow$ yükseklik maliyeti max

$\log_3(n+1)-1$ - en iyi durumda
yükseklik

